Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А.

Проверил:

Никитин С. К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода

Ход работы:

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной).

2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно.

3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы.

4. Выполните пункты 1-3 для датасетов из ЛР 2.

5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№	Выборка	Тип задачи	Целевая переменная
8	https://archive.ics.uci.edu/dataset/162/forest+fires	регрессия	area

Код программы:

```
import copy
import os
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from IPython.display import display
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (mean_absolute_error, mean_squared_error, r2_score,
                             accuracy_score, f1_score, confusion_matrix,
                             ConfusionMatrixDisplay)

device = "cuda" if torch.cuda.is_available() else "cpu"
print("device:", device)

HIDDEN = [64, 32, 16, 8]
ACT = nn.ReLU
PRETRAIN_EPOCHS = 50
PRETRAIN_LR = 1e-3
HEAD_EPOCHS = 50
FINETUNE_EPOCHS = 200
```

```
LR = 1e-3
BATCH = 32
SEEDS = [0, 1, 2, 3, 4]


def T(a):
    return torch.tensor(np.asarray(a), dtype=torch.float32, device=device)

def make_mlp(in_dim, hidden, out_dim):
    layers, d = [], in_dim
    for h in hidden:
        layers += [nn.Linear(d, h), ACT()]
        d = h
    layers.append(nn.Linear(d, out_dim))
    return nn.Sequential(*layers).to(device)

def batches(n, batch):
    idx = torch.randperm(n, device=device)
    for i in range(0, n, batch):
        yield idx[i:i + batch]

def train_supervised(model, Xtr, ytr, Xva, yva, loss_fn, epochs, lr, params=None):
    opt = torch.optim.Adam(model.parameters() if params is None else params, lr=lr)
    best, best_state = float("inf"), None
    hist = {"train": [], "val": []}
    for ep in range(epochs):
        total = 0.0
        for idx in batches(len(Xtr), BATCH):
            opt.zero_grad()
            loss = loss_fn(model(Xtr[idx]), ytr[idx])
            loss.backward()
            opt.step()
            total += loss.item() * len(idx)
        with torch.no_grad():
            val = loss_fn(model(Xva), yva).item()
        hist["train"].append(total / len(Xtr))
        hist["val"].append(val)
        if val < best:
            best, best_state = val, copy.deepcopy(model.state_dict())
    model.load_state_dict(best_state)
    return hist

@torch.no_grad()
def predict(model, X):
    return model(X).squeeze(1).cpu().numpy()


def pretrain_layers(model, Xtr, Xva):
    linears = [m for m in model if isinstance(m, nn.Linear)]
    h_tr, h_va = Xtr, Xva
    for enc in linears[:-1]:
        in_d, out_d = enc.in_features, enc.out_features
        ae = nn.Sequential(enc, ACT(), nn.Linear(out_d, in_d)).to(device)
        train_supervised(ae, h_tr, h_tr, h_va, h_va, nn.MSELoss(),
                         PRETRAIN_EPOCHS, PRETRAIN_LR)
        with torch.no_grad():
            h_tr, h_va = ACT()(enc(h_tr)), ACT()(enc(h_va))

def train_head(model, Xtr, ytr, Xva, yva, loss_fn):
    last = [m for m in model if isinstance(m, nn.Linear)][-1]
```

```
    train_supervised(model, Xtr, ytr, Xva, yva, loss_fn,
                     HEAD_EPOCHS, LR, params=last.parameters())


def reg_metrics(y_log_true, y_log_pred):
    a_true = np.expm1(y_log_true)
    a_pred = np.clip(np.expm1(y_log_pred), 0, None)
    m = a_true > 0
    return {"MAE": mean_absolute_error(a_true, a_pred),
            "RMSE": float(np.sqrt(mean_squared_error(a_true, a_pred))),
            "MAPE_%": float(np.mean(np.abs((a_true[m] - a_pred[m]) / a_true[m])) * 100),
            "R2_log": r2_score(y_log_true, y_log_pred)}

def evaluate(task, y_true, out):
    if task == "reg":
        return reg_metrics(y_true, out), None
    pred = (out > 0).astype(int)
    return ({"Accuracy": accuracy_score(y_true, pred),
             "F1_macro": f1_score(y_true, pred, average="macro")},
            confusion_matrix(y_true, pred))

def run_experiment(title, X, y, task, class_names=None):
    loss_fn = nn.MSELoss() if task == "reg" else nn.BCEWithLogitsLoss()
    rows, curves, cms = [], {}, {}
    for seed in SEEDS:
        strat = y if task == "clf" else None
        X_tr, X_tmp, y_tr, y_tmp = train_test_split(
            X, y, test_size=0.30, random_state=seed, stratify=strat)
        X_va, X_te, y_va, y_te = train_test_split(
            X_tmp, y_tmp, test_size=0.50, random_state=seed,
            stratify=y_tmp if task == "clf" else None)

        sc = StandardScaler().fit(X_tr)
        Xtr, Xva, Xte = [T(sc.transform(a)) for a in (X_tr, X_va, X_te)]
        ytr, yva = T(y_tr).unsqueeze(1), T(y_va).unsqueeze(1)

        for mode in ["без предобучения", "с предобучением"]:
            torch.manual_seed(seed)
            model = make_mlp(Xtr.shape[1], HIDDEN, 1)
            if mode == "с предобучением":
                pretrain_layers(model, Xtr, Xva)
                train_head(model, Xtr, ytr, Xva, yva, loss_fn)
            hist = train_supervised(model, Xtr, ytr, Xva, yva, loss_fn, FINETUNE_EPOCHS, LR)
            metrics, cm = evaluate(task, y_te, predict(model, Xte))
            rows.append({"режим": mode, "seed": seed, **metrics})
            if seed == SEEDS[0]:
                curves[mode], cms[mode] = hist["val"], cm
        print(f"seed {seed} готов")

    df = pd.DataFrame(rows)
    print(f"\n=== {title}: среднее и std по {len(SEEDS)} запускам, тестовая выборка ===")
    display(df.drop(columns="seed").groupby("режим").agg(["mean", "std"]).round(3))

    plt.figure(figsize=(6, 4))
    for mode, v in curves.items():
        plt.plot(v, label=mode)
    plt.xlabel("эпоха основного обучения"); plt.ylabel("loss на val")
    plt.title(title); plt.legend(); plt.grid(alpha=.3); plt.show()

    if task == "clf":
```

```
        fig, axs = plt.subplots(1, 2, figsize=(8, 3.5))
        for ax, (mode, cm) in zip(axs, cms.items()):
            ConfusionMatrixDisplay(cm, display_labels=class_names).plot(ax=ax, colorbar=False)
            ax.set_title(mode)
        plt.tight_layout(); plt.show()
    return df


ff = pd.read_csv("forestfires.csv")
print(ff.shape)
display(ff.head(3))

area = ff["area"]
feat = ff.drop(columns="area")
print("доля пожаров с area == 0:", round(float((area == 0).mean()), 2))

X_ff = pd.get_dummies(feat, columns=["month", "day"], dtype=float).to_numpy(dtype=np.float32)
y_ff = np.log1p(area.to_numpy())

df_ff = run_experiment("Forest Fires (регрессия)", X_ff, y_ff, task="reg")


RICE_FILE = "Rice_Cammeo_Osmancik.arff"

ext = os.path.splitext(RICE_FILE)[1].lower()
if ext in (".xlsx", ".xls"):
    rice = pd.read_excel(RICE_FILE)
elif ext == ".arff":
    from scipy.io import arff
    data, meta = arff.loadarff(RICE_FILE)
    rice = pd.DataFrame(data)
else:
    rice = pd.read_csv(RICE_FILE, sep=None, engine="python")

print(rice.shape)
display(rice.head(3))

feat = rice.iloc[:, :-1]
target = rice.iloc[:, -1]
if isinstance(target.iloc[0], bytes):
    target = target.str.decode("utf-8")
target = target.astype(str).str.strip()

classes = sorted(target.unique())
print(classes, target.value_counts().to_dict())

X_rice = feat.to_numpy(dtype=np.float32)
y_rice = (target == classes[1]).astype(int).to_numpy()

df_rice = run_experiment("Rice (классификация)", X_rice, y_rice, task="clf",
class_names=classes)
```

Результат программы:

```
device: cuda
```

```
(517, 13)
```

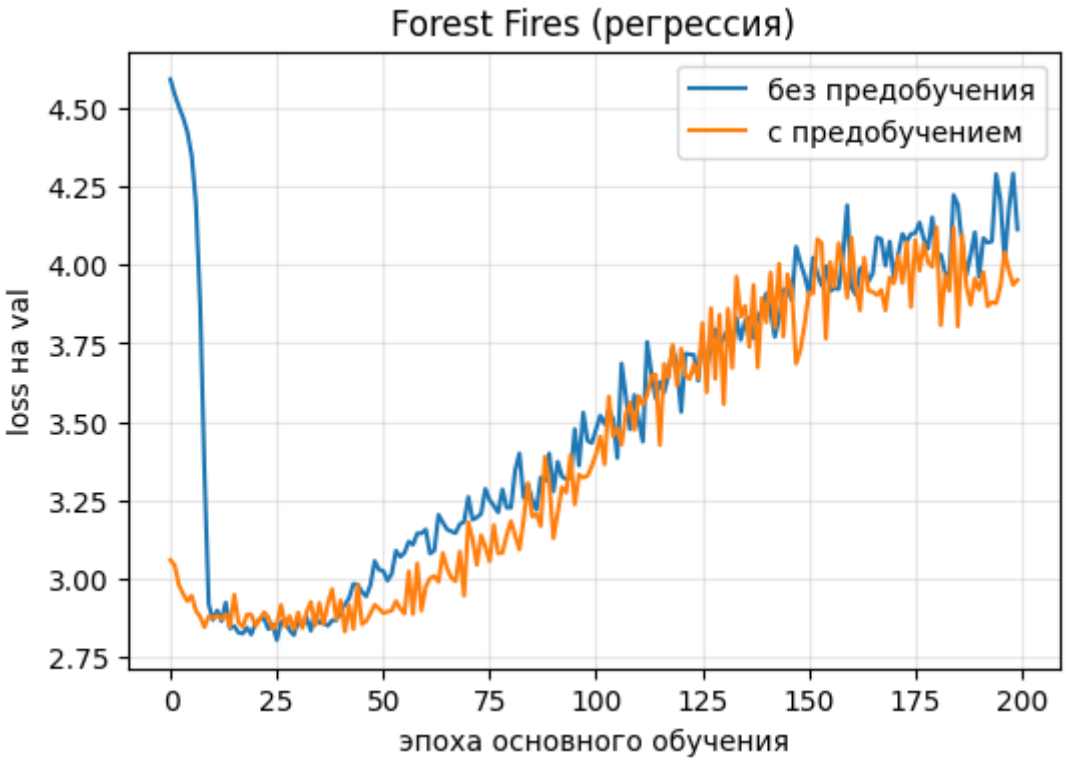
X	Y	month	day	FFMC	DMC	DC	ISI	temp	RH	wind	rain	area
7	5	mar	fri	86.2	26.2	94.3	5.1	8.2	51	6.7	0.0	0.0
7	4	oct	tue	90.6	35.4	669.1	6.7	18.0	33	0.9	0.0	0.0
7	4	oct	sat	90.6	43.7	686.9	6.7	14.6	33	1.3	0.0	0.0

```
доля пожаров с area == 0: 0.48
seed 0 готов
seed 1 готов
seed 2 готов
seed 3 готов
seed 4 готов

=== Forest Fires (регрессия): среднее и std по 5 запускам, тестовая выборка
===
```

	MAE		RMSE		MAPE_%		R2_log	
Режим	**mean**	**std**	**mean**	**std**	**mean**	**std**	**mean**	**std**
без предобучения	13.095	4.841	50.547	35.923	107.835	29.442	-0.069	0.047
с предобучением	420.021	912.879	3627.519	8018.522	6934.382	15273.687	-0.190	0.256

График forest + fires с предобучением и без предобучения:



```
(3810, 8)
```

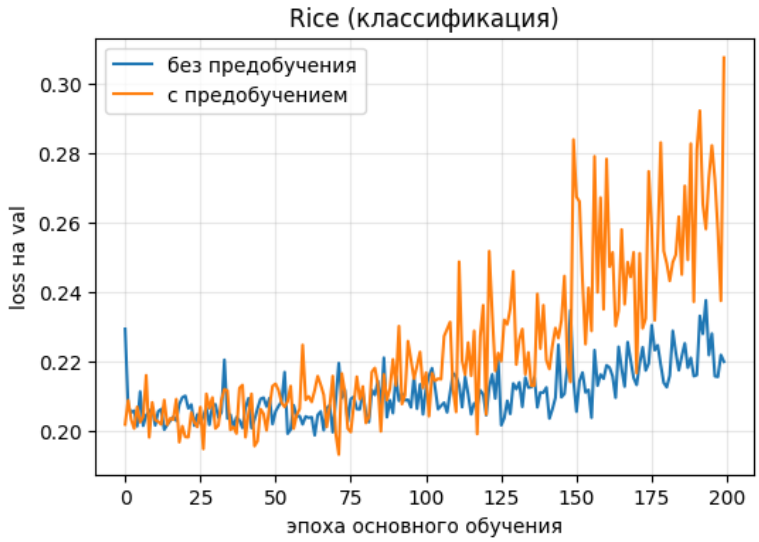
Area	Perimeter	Major_Axis_Length	Minor_Axis_Length	Eccentricity	Convex_Area	Extent	Class
15231.0	525.578979	229.749878	85.093788	0.928882	15617.0	0.572896	b'Cammeo'
14656.0	494.311005	206.020065	91.730972	0.895405	15072.0	0.615436	b'Cammeo'
14634.0	501.122009	214.106781	87.768288	0.912118	14954.0	0.693259	b'Cammeo'

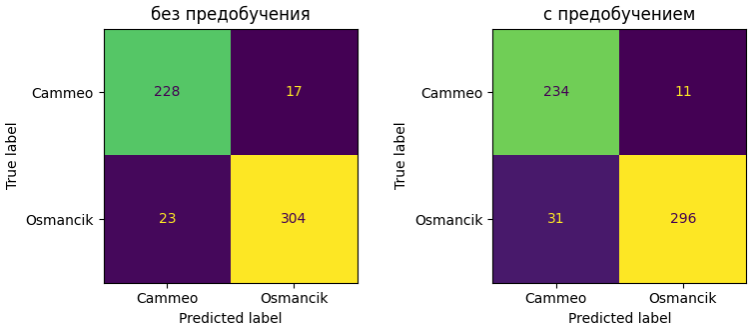
```
['Cammeo', 'Osmancik'] {'Osmancik': 2180, 'Cammeo': 1630}
seed 0 готов
seed 1 готов
seed 2 готов
seed 3 готов
seed 4 готов

=== Rice (классификация): среднее и std по 5 запускам, тестовая выборка ===
```

	Accuracy		F1_macro	
Режим	**mean**	**std**	**mean**	**std**
без предобучения	0.929	0.006	0.928	0.006
с предобучением	0.926	0.001	0.925	0.001

График Rice + Cammeo + Osmancik с предобучением и без предобучения:





Предобучение автоэнкодерами на этих данных не улучшило результат: на Rice точность одинаковая (около 93%), а на Forest Fires оно не помогло и сделало результаты менее стабильными, потому что сеть небольшая и обычного обучения ей хватает.

Вывод: Проделав данную лабораторную работу я научился осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода